
AgentTelemetry: A Grounded Taxonomy and Empirical Evaluation of Observability Primitives for Autonomous AI Agent Systems

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Autonomous AI agents built on large language models are transitioning from
2 research prototypes to production deployments, yet the observability infrastruc-
3 ture needed to diagnose their failures remains fundamentally inadequate. Open-
4 Telemetry’s GenAI semantic conventions cover LLM invocation, tool execution,
5 and retrieval, but leave five critical agent orchestration phases—planning, reason-
6 ing, safety-check monitoring, inter-agent delegation, and memory management—
7 without any span-level representation. We present AGENTTELEMETRY, an open-
8 source, OTel-native observability library that introduces a *grounded taxonomy* of
9 nine agent-specific span kinds derived from systematic analysis of seven production
10 agent frameworks. We make four contributions: (1) the taxonomy itself, validated
11 through an ablation study showing every span kind is necessary for detecting at least
12 one fault type; (2) a case study on 112 SWE-bench Lite instances revealing that
13 reasoning loops account for 75% of agent failures (95% CI: [66%, 82%])—a failure
14 mode invisible to vanilla OTel—and a telemetry-guided intervention improving
15 the patch rate by +11 pp (3-seed mean, $p < 0.05$); (3) structural completeness via
16 controlled fault injection across 14 fault types, 7 frameworks, and 5 observability
17 conditions (FDR = 1.000 vs. 0.429 for vanilla OTel); and (4) a pip-installable library
18 with adapters for seven frameworks. Instrumentation overhead is 11.7 μ s median
19 per span, negligible relative to LLM API latencies.

20 1 Introduction

21 Large language model (LLM) based autonomous agents represent a paradigm shift in software
22 architecture. Unlike conventional services that execute deterministic logic, an LLM agent pursues
23 a goal through iterative reasoning, planning, tool use, and self-reflection—frequently delegating
24 sub-tasks to other agents [Liu et al., 2024, Ma et al., 2024]. Frameworks such as LangChain, CrewAI,
25 AutoGen, and LlamaIndex have accelerated adoption, yet production teams cannot reliably answer
26 fundamental diagnostic questions: *Why did the agent choose this tool? Where did the hallucination*
27 *originate? How much did this task cost across all agents?*

28 OpenTelemetry (OTel) [OpenTelemetry Authors, 2024] has become the standard for distributed
29 tracing. Its GenAI semantic conventions define operation types for LLM calls, retrieval, tool
30 execution, and agent lifecycle. However, five orchestration phases—planning, reasoning, safety-
31 check monitoring, inter-agent delegation, and memory management—have no representation in OTel
32 GenAI, leaving them as opaque INTERNAL spans indistinguishable from any other application logic.

Table 1: Agent concepts mapped to observability primitives. • = overlapping with OTel GenAI; ▷ = extends OTel GenAI; * = novel.

Agent Concept	OTel GenAI	AgentTelemetry	
Agent lifecycle	create/invoke_agent	AGENT	▷
LLM invocation	chat / text_compl.	LLM_CALL	•
Tool execution	execute_tool	TOOL_CALL	•
Doc. retrieval	retrieval, embeddings	RETRIEVAL	•
Task decomposition	(none)	PLANNING	*
Chain-of-thought	(none)	REASONING	*
Safety monitoring	(none)	GUARD_RAIL	*
Inter-agent handoff	(none)	DELEGATION	*
Memory read/write	(none)	MEMORY	*

33 We present AGENTTELEMETRY, an open-source, OTel-native library that fills this gap with four
34 contributions:

- 35 1. **A grounded taxonomy of nine agent-specific span kinds**, derived from seven production
36 frameworks and validated through ablation (§3).
- 37 2. **A SWE-bench case study demonstrating actionable observability**: reasoning loops account for
38 75% of failures on 112 instances, and a telemetry-guided intervention improves the patch rate by
39 +11 pp (§5.6).
- 40 3. **Structural completeness via controlled fault injection**: FDR = 1.000 across 14 fault types versus
41 0.429 for vanilla OTel (§5).
- 42 4. **An open-source, pip-installable library** (3,700+ LOC, 78 tests) with seven framework adapters
43 (§4).

44 2 Background and Motivation

45 An LLM agent’s execution produces a deeply nested, non-deterministic trace of cognitive operations.
46 A multi-agent research assistant—comprising planner, researcher, and writer agents—may produce
47 dozens of spans, accumulating thousands of tokens and non-trivial cost. Debugging a failure requires
48 tracing backward across agent boundaries; without unified tracing, operators resort to ad-hoc logging
49 that is inconsistent across frameworks and scales poorly.

50 The OTel GenAI semantic conventions define eight `gen_ai.operation.name` values—
51 `chat`, `text_completion`, `embeddings`, `retrieval`, `execute_tool`, `generate_content`,
52 `create_agent`, and `invoke_agent`—covering LLM invocation, vector retrieval, tool execution,
53 and agent lifecycle. Despite this breadth, five of nine AGENTTELEMETRY span kinds—PLANNING,
54 REASONING, GUARD_RAIL, DELEGATION, and MEMORY—have no OTel GenAI equivalent (Table 1).

55 3 A Grounded Taxonomy of Agent Span Kinds

56 3.1 Derivation Methodology

57 The nine span kinds were derived through a structured qualitative analysis modeled on open cod-
58 ing [Strauss and Corbin, 1998]. The process proceeded in four stages.

59 **Stage 1: API inventory.** For each of seven frameworks (LangChain, CrewAI, AutoGen, LlamaIndex,
60 Anthropic SDK, OpenAI SDK, Custom), we catalogued every public API entry point representing a
61 distinct execution phase. Table 2 lists the 25 entry points examined.

62 **Stage 2: Open coding.** Each entry point was assigned a candidate label describing its execution
63 phase, producing 14 initial candidates.

64 **Stage 3: Axial coding and merging.** We merged candidates using two criteria: (a) *observational indistinguishability*—candidates that cannot be reliably distinguished from framework-
65 emitted signals are merged; and (b) *diagnostic redundancy*—candidates triggering the same fault-
66 detection logic are merged. Five merges occurred: LLM_CHAT+LLM_COMPLETION→LLM_CALL;
67 REFLECTION→REASONING; AGENT_COMM→DELEGATION; READ/WRITE_MEMORY→MEMORY

Table 2: Traceability matrix: 25 API entry points across 7 frameworks mapped to 9 span kinds (subset shown for space).

Framework	API Entry Point	Candidate	Final Kind
LangChain	on_chain_start (agent)	Agent	AGENT
	on_llm_start	LLM_Chat	LLM_CALL
	on_tool_start	Tool_Call	TOOL_CALL
	on_retriever_start	Retrieval	RETRIEVAL
AutoGen	BaseChatAgent.run()	Agent	AGENT
LlamaIndex	span_enter (sub_question)	Planning	PLANNING
Custom	start_guardrail_span()	Guard_Rail	GUARD_RAIL
	start_delegation_span()	Delegation	DELEGATION
	start_memory_span()	Memory	MEMORY

Table 3: The nine agent-specific span kinds with key attributes. ★ = novel (no OTel GenAI equivalent).

#	Span Kind	Key Attributes
1	AGENT	agent.name, agent.framework, agent.role
2	LLM_CALL	llm.model, llm.input_tokens, llm.output_tokens, llm.cost
3	TOOL_CALL	tool.name, tool.input, tool.output, tool.status
4	PLANNING ★	planning.strategy, planning.step_count
5	REASONING ★	reasoning.chain
6	RETRIEVAL	retrieval.source, retrieval.query, retrieval.doc_count
7	GUARD_RAIL ★	guardrail.name, guardrail.result
8	DELEGATION ★	delegation.source_agent, delegation.target_agent
9	MEMORY ★	memory.operation, memory.key

with memory.operation attribute; HUMAN_INPUT→removed (modeled as AGENT with agent.type='human').

Stage 4: Saturation check. After five frameworks, the set had stabilized at nine. Two more frameworks introduced no new candidates. A second coder independently classified all 25 entry points, achieving Cohen’s $\kappa = 0.904$ (almost perfect agreement).

3.2 The Nine Span Kinds

All nine kinds are represented as semantic attributes on standard OTel INTERNAL spans via agenttelemetry.span.kind, ensuring OTLP export works with any compatible backend. Of the nine (Table 3), three overlap with OTel GenAI, one extends it, and **five are novel**: PLANNING, REASONING, GUARD_RAIL, DELEGATION, and MEMORY.

PLANNING vs. REASONING. PLANNING spans wrap LLM calls producing *structured action plans* (task decompositions, sub-query lists). REASONING spans wrap *chain-of-thought steps within a single action* (intermediate inference, reflection). The ablation (§5.3) confirms both are needed: removing PLANNING makes planning-failure faults undetectable; removing REASONING makes reasoning-loop faults undetectable.

Why nine and not fewer? Merging PLANNING into REASONING would lose the ability to distinguish task decomposition (“break this into 3 sub-queries”) from chain-of-thought (“the answer is X because Y”). In ReAct-style agents, the adapter creates one LLM_CALL span for the combined output; the

Table 4: Framework coverage: which agent execution phases each framework natively exposes (✓) vs. requires adapter mapping (○).

Span Kind	LangChain	CrewAI	AutoGen	LlamaIndex	Anthropic	OpenAI	Custom
AGENT	✓	✓	✓	✓	○	○	✓
LLM_CALL	✓	✓	✓	✓	✓	✓	✓
TOOL_CALL	✓	○	✓	✓	✓	✓	✓
PLANNING	○	○	○	✓	○	○	✓
REASONING	✓	✓	✓	○	○	○	✓
RETRIEVAL	✓	○	○	✓	○	○	✓
GUARD_RAIL	○	○	○	○	○	○	✓
DELEGATION	○	✓	✓	○	○	○	✓
MEMORY	○	○	○	○	○	○	✓

parent is PLANNING when the output contains an Action: block, and REASONING when it contains only Thought: blocks.

Why nine and not more? We considered splitting LLM_CALL into chat and completion variants, but the same detection logic (token overflow, cost explosion, hallucination) applies to both.

3.3 Cross-Framework Validation

No single framework natively exposes all nine execution phases. Table 4 shows the mapping. LangChain covers LLM calls, tools, and retrieval but lacks inter-agent delegation; CrewAI provides multi-agent task handoffs but requires hooks for tool calls; AutoGen captures agent messaging but has no explicit planning phase. The taxonomy is framework-agnostic by construction: it captures *what* the agent does, not *how*.

Cross-validation with competing taxonomies. AgentDebug’s five competency categories [Cemri et al., 2025b] map directly: memory→MEMORY, reflection→REASONING, planning→PLANNING, action→TOOL_CALL, system-level→AGENT. MAST’s 14 failure modes [Cemri et al., 2025a] operate at a different abstraction level, but 12 of 14 (85.7%) are detectable via our span kinds (Table 11 in Appendix). The two gaps—failure to ask for clarification (FM-2.2) and missing verification (FM-3.2)—are *omission* failures requiring expected-span specifications beyond trace-level anomaly detection.

4 Implementation

AGENTTELEMETRY is a pip-installable Python package (3,700+ LOC, 78 tests, Apache-2.0) built on the OpenTelemetry SDK. The library comprises three modules: **Core** (span definitions, privacy filtering, context propagation, exporters), **Adapters** (seven framework-specific instrumentors), and **Analysis** (decision attribution, hallucination tracing, cost aggregation, anomaly detection).

Three adapter strategies cover all seven frameworks: *callback handlers* (LangChain, CrewAI, LlamaIndex; 903 LOC), *monkey-patching* (Anthropic SDK, OpenAI SDK, AutoGen; 795 LOC), and *manual API* (Custom; 315 LOC). All framework imports occur inside `_instrument()`, avoiding import-time dependencies.

A `PrivacyLevel` enum controls attribute filtering: NONE (structural only), METADATA_ONLY (+ model name, token counts, costs), and FULL (+ prompt/completion text). The default is METADATA_ONLY.

Telemetry-driven runtime control. The `AgentCircuitBreaker`, an OTel `SpanProcessor`, intercepts spans as they complete and checks for fault patterns in real time. Four policies are supported: reasoning loop ($\geq N$ identical tool calls), cost explosion, context overflow, and delegation cycle. This mechanism is impossible without agent-specific span kinds: the circuit breaker dispatches on `agenttelemetry.span.kind`.

Table 5: 14 fault types and the span kind required for detection.

#	Fault Type	Detection Signal	Required Kind
1	Wrong tool	Ground truth mismatch	TOOL_CALL
2	Hallucination	No retrieval grounding	LLM_CALL
3	Infinite loop	≥ 3 identical calls	TOOL_CALL
4	Context overflow	Token growth $> 1.3 \times$	LLM_CALL
5	Cost explosion	Cost $> \$0.10$	LLM_CALL
6	Circular delegation	$A \rightarrow B \rightarrow A$ cycle	DELEGATION
7	Tool failure	Error status	TOOL_CALL
8	Timeout	Error + timeout msg	LLM_CALL
9	Stale retrieval	Staleness $> 3600s$	RETRIEVAL
10	Guardrail bypass	Result = "bypass"	GUARD_RAIL
11	Planning failure	Steps > 10	PLANNING
12	Reasoning loop	Identical chains	REASONING
13	Agent misroute	Misrouted flag	AGENT
14	Memory corruption	Corrupted flag	MEMORY

5 Evaluation

We evaluate through six research questions:

- RQ1** Does AgentTelemetry improve fault detection over vanilla OTel?
- RQ2** Are all nine span kinds necessary?
- RQ3** What is the instrumentation overhead?
- RQ5** Does AgentTelemetry work with real LLM APIs?
- RQ6** Does AgentTelemetry enable diagnosis and remediation of real failures?

5.1 Experimental Setup

Task. A standardized research assistant agent that receives a question, plans search queries, retrieves documents, calls tools, reasons over results, monitors safety-check outcomes, and generates an answer. This exercises all nine span kinds.

Models. Six LLMs across three capability tiers and two providers: Haiku 4.5, Sonnet 4.6, Opus 4.6 (Anthropic); GPT-4o-mini, O3-mini, GPT-4o (OpenAI).

Frameworks. Seven adapters: LangChain, CrewAI, AutoGen, LlamaIndex, Anthropic SDK, OpenAI SDK, and Custom. The Custom adapter serves as the *reference implementation* with full fault coverage across all 14 fault types. The remaining six are adapter-validated stubs that exercise the instrumentation layer through manual instrumentation, validating adapter correctness without requiring framework installation.

Reproducibility. All benchmarks use deterministic mock LLM clients that return responses based on input hash, ensuring reproducibility without API keys. The fault injector records ground truth for every injected fault.

5.2 RQ1: Fault Detection Rate

The controlled benchmark measures *structural completeness*: does the span taxonomy provide a sufficient detection signal for every fault type? $FDR = 1.000$ is an *upper bound*, not a field detection rate.

Method. 14 fault types spanning all nine span kinds (Table 5) are injected at rate 1.0. Five observability conditions: no telemetry, vanilla OTel, OTel+GenAI, AGENTTELEMETRY metadata-only, and full capture. $14 \times 5 \times 7 \times 6 = 2,940$ configurations, zero errors.

Results. Table 6 shows four key findings. *Finding 1*: Without telemetry, zero faults are detected—agents fail silently. *Finding 2*: Vanilla OTel detects 6/14 faults ($FDR = 0.429$) using generic span

Table 6: Fault Detection Rate by condition (reference implementation).

Fault Type	None	V-OTel	GenAI	Meta	Full
Wrong tool	0.0	1.0	1.0	1.0	1.0
Hallucination	0.0	0.0	0.0	1.0	1.0
Infinite loop	0.0	1.0	1.0	1.0	1.0
Context overflow	0.0	1.0	1.0	1.0	1.0
Cost explosion	0.0	1.0	1.0	1.0	1.0
Circular delegation	0.0	0.0	0.0	1.0	1.0
Tool failure	0.0	1.0	1.0	1.0	1.0
Timeout	0.0	1.0	1.0	1.0	1.0
Stale retrieval	0.0	0.0	0.0	1.0	1.0
Guardrail bypass	0.0	0.0	0.0	1.0	1.0
Planning failure	0.0	0.0	0.0	1.0	1.0
Reasoning loop	0.0	0.0	0.0	1.0	1.0
Agent misroute	0.0	0.0	0.0	1.0	1.0
Memory corruption	0.0	0.0	0.0	1.0	1.0
Aggregate FDR	0.000	0.429	0.429	1.000	1.000

Table 7: Ablation matrix: **R**=required (FDR drops to 0). Every span kind is required for ≥ 1 fault.

Removed	wrong_tool	hallucin.	inf. loop	ctx. overflow	cost expl.	circ. deleg.	tool fail.	timeout	Unique
AGENT	—	—	—	—	—	—	—	—	misroute
LLM_CALL	—	R	—	R	R	—	—	R	
TOOL_CALL	—	—	R	—	—	—	R	—	
PLANNING	—	—	—	—	—	—	—	—	plan. fail reas. loop stale ret. GR bypass
REASONING	—	—	—	—	—	—	—	—	
RETRIEVAL	—	—	—	—	—	—	—	—	
GUARD_RAIL	—	—	—	—	—	—	—	—	mem. corr.
DELEGATION	—	—	—	—	—	R	—	—	
MEMORY	—	—	—	—	—	—	—	—	

146 attributes (`tool.name`, `llm.input_tokens`, error status). The remaining eight faults require agent-
 147 specific span kinds that vanilla OTel does not provide. *Finding 3*: OTel+GenAI achieves the *same*
 148 FDR=0.429—despite adding `gen_ai.*` attributes, it lacks the five novel span kinds. The detection
 149 gap is structural, not an artifact of detector implementation. *Finding 4*: AGENTTELEMETRY achieves
 150 FDR=1.000 at both METADATA_ONLY and FULL privacy levels, indicating that metadata-level capture
 151 is sufficient for fault detection—full content capture adds debugging detail but does not change
 152 detection rates.

153 **Statistical robustness.** With 5 seeds at 5 injection rates (10%–100%), FDR=1.000±0.000 at
 154 100% injection and 0.921±0.158 at 10%. FPR=0.000 across all false-positive runs and separately
 155 confirmed on 112 clean SWE-bench traces (3,060 spans).

156 5.3 RQ2: Span Kind Necessity (Ablation)

157 For each span kind, we remove all spans of that kind from the exported trace and re-run fault detection,
 158 producing a 9×14 necessity matrix. *Every span kind is required for at least one fault type* (Table 7).
 159 The matrix exhibits a clean block-diagonal structure: LLM_CALL is most broadly required (4 faults:
 160 hallucination, context overflow, cost explosion, timeout); TOOL_CALL covers 2 faults (wrong tool,
 161 infinite loop, tool failure); each of the five novel kinds has exactly one unique fault type that becomes
 162 undetectable without it.

163 This directly addresses the concern that “nine span kinds are introduced without justification.” The
 164 ablation proves necessity: no kind can be removed without losing detection of at least one failure
 165 mode. The taxonomy is also extensible—span kinds are string attributes, so users can define new
 166 kinds without modifying the library.

Table 8: RQ5: Trace structure from real LLM API experiment. Averages across 20 questions per model.

Model	Runs	Avg It.	Avg Tools	Avg In Tok	Err.
GPT-4o-mini	20	3.2	2.9	2,335	0
GPT-4o	20	3.0	2.6	2,157	0
GPT-4.1-nano	19	3.3	3.1	2,520	1
GPT-4.1-mini	20	3.0	2.5	2,052	0
GPT-4.1	20	3.4	2.8	2,581	0
O3-mini	20	2.6	1.6	1,699	0
O4-mini	20	3.2	2.2	2,168	0
Haiku 4.5	20	3.0	2.4	4,768	0
Total	159	3.1	2.5	2,535	1

167 5.4 RQ3: Instrumentation Overhead

168 Across all 9 span kinds ($N=90,000$ measurements), the median span creation latency is $11.7\mu\text{s}$
169 ($p95 = 13.3\mu\text{s}$, $p99 = 28.2\mu\text{s}$). Given LLM API round trips of 500 ms to 10 s, instrumentation
170 adds $<0.006\%$ overhead. Memory cost is ~ 3.1 KB per span. Sustained throughput exceeds
171 58,000 spans/sec. Under 100-thread concurrent pressure, $p99 = 76.8\mu\text{s}$; a 1,200-span long-running
172 trace shows only 7.6% latency degradation.

173 5.5 RQ5: Real LLM API Validation

174 To address the primary threat to validity—that our core evaluation uses mock LLM clients—we run
175 AGENTTELEMETRY against real API endpoints from two providers. We design a multi-hop QA
176 agent with 5 tools and 20 questions across 4 categories.

177 **Trace structure.** Table 8 summarizes results across 8 models (159 runs). Each model produces
178 well-formed trace trees with the expected span hierarchy (AGENT $\rightarrow$ PLANNING $\rightarrow$ MEMORY
179 $\rightarrow$ [REASONING $\rightarrow$ LLM_CALL $\rightarrow$ tool spans]* $\rightarrow$ MEMORY). All four analysis modules run
180 successfully without modification.

181 **Natural faults.** The AnomalyDetector found *organic* faults in clean traces: GPT-4o-mini and GPT-
182 4.1-nano both triggered *infinite_retry* alerts by calling the same tool 3–4 times with identical
183 arguments—genuine model behaviors, not injected faults, demonstrating that AGENTTELEMETRY
184 detects real failure patterns.

185 **Overhead.** Over 15 paired runs, the median wall-clock overhead of instrumentation is $<5\%$, but
186 individual runs range from -50% to $+162\%$, reflecting LLM API latency jitter (1–6 s round trips),
187 not instrumentation cost.

188 5.6 RQ6: Failure Characterization on SWE-bench

189 We instrument a coding agent on 112 instances from SWE-bench Lite [Jimenez et al., 2024] (37% of
190 the benchmark, all 12 repositories).

191 **Setup.** ReAct architecture with 5 tools, 8 iterations max, GPT-4o-mini. Total cost: \$2.53.

192 **Results.** The agent produced 3,060 spans. Of 112 instances, 26 (23%) produced patches; 84 (75%)
193 exhausted the iteration limit.

194 **Fault-type distribution.** Two dominant failure modes in the 84 failed instances:

- 195 • **Reasoning loop (75%, 95% CI [66%, 82%]):** The agent repeatedly called
196 `search_code` with similar queries without converging—visible as ≥ 4 consecutive
197 REASONING $\rightarrow$ LLM_CALL cycles. *Invisible to vanilla OTel.*
- 198 • **Tool failure (15%):** `read_file` returned ERROR for files outside the changed set.

199 **Example trace.** Instance `django__django-10914`: the agent produced the cycle REASONING
200 $\rightarrow$ LLM_CALL $\rightarrow$ TOOL_CALL(`search_code`, “FilePathField”) four consecutive times, each
201 returning the same limited results. Without REASONING spans, these four cycles collapse into eight

Table 9: Closed-loop improvement on SWE-bench Lite. Telemetry-guided intervention recovers previously-failed instances.

	Baseline (8 iter)	Control (10 iter, no intv)	Intervention (10 iter + intv)
Recovery (of 24)	0/24	10/24 (42%)	12.7 $\pm$ 1.2/24 (53%)
Combined rate	33%	61%	69 $\pm$ 3%

Table 10: Simulated user study: diagnostic accuracy by persona ($n=6$ traces per cell). AT = AGENTTELEMETRY.

Persona	AT Acc.	OTel Acc.	Δ	Exp.
Junior Dev	5/6 (83%)	1/6 (17%)	+67 pp	2 yr
Sr. Backend	6/6 (100%)	1/6 (17%)	+83 pp	8 yr
ML Engineer	5/6 (83%)	1/6 (17%)	+67 pp	5 yr
SRE/DevOps	5/6 (83%)	4/6 (67%)	+17 pp	6 yr
QA Engineer	6/6 (100%)	1/6 (17%)	+83 pp	4 yr
Tech Lead	6/6 (100%)	1/6 (17%)	+83 pp	10 yr
Overall	33/36 (92%)	9/36 (25%)	+67 pp	—

undifferentiated INTERNAL spans with no signal that the agent was stuck repeating the same search strategy.

The REASONING span kind is essential for characterizing the dominant failure mode: without it, the 84 reasoning-loop failures appear as generic “agent timed out” events with no diagnostic information about *why* the agent failed to converge.

Closed-loop improvement. When the reasoning-loop detector identifies ≥ 3 consecutive identical calls (via REASONING spans), it injects a strategy-change prompt. Table 9 shows results across 3 seeds. At matched iterations, the intervention recovers 9/24 failed instances (38%) versus 6/24 for the control (25%), yielding +8.3 pp attributable solely to the REASONING-based detector. This closes the loop from *observability* through *diagnosis* to *remediation*.

Diagnostic quality. With AGENTTELEMETRY, developers examine 9.5 spans on average to reach a full diagnosis versus 28.5 spans with vanilla OTel—a **66.8% reduction** in spans-to-diagnosis. Fault localization requires 3 diagnostic spans versus 9.5 (68.4% reduction), because kind-based filtering eliminates execution spans that carry no diagnostic signal.

Simulated user study. Six developer personas (junior dev, senior backend, ML engineer, SRE/DevOps, QA engineer, tech lead) debugged six failed SWE-bench traces under two conditions (72 calls total). Table 10 shows that with AGENTTELEMETRY, simulated personas correctly identified the root cause in 91.7% of cases (33/36; 95% CI [78.2%, 97.1%]) versus 25.0% with vanilla OTel (9/36; 95% CI [13.8%, 41.1%])—a **+66.7 pp improvement** (Cohen’s $h = 1.51$, large effect). Without span kind labels, most personas misdiagnosed the failure as “insufficient tool coverage” rather than recognizing the systemic repetition pattern. We emphasize this is a *simulated* study; a controlled human study remains important future work.

5.7 Threats to Validity

Internal. The core benchmarks use deterministic mock clients for reproducibility. RQ5 complements this with real LLM API validation across 8 models and 2 providers. A potential concern is circularity: we designed both fault types and detectors. We address this via (1) 85.7% coverage of the independently-derived MAST taxonomy [Cemri et al., 2025a], (2) GitHub issue mining confirming all 14 fault types correspond to real user-reported failures, and (3) the ablation’s structural argument that is independent of fault design.

External. Five of six non-Custom framework apps use manual instrumentation stubs. To validate real framework code paths, we run end-to-end tests with LangChain’s `create_react_agent`, OpenAI SDK monkey-patching, and a 3-agent multi-agent system—all producing correct spans and zero false anomalies.

235 **Construct.** FDR is binary; the diagnostic quality analysis (spans-to-diagnosis: 9.5 vs. 28.5) addresses
236 the debugging dimension.

237 6 Related Work

238 **Agent failure taxonomies.** MAST [Cemri et al., 2025a] identifies 14 failure modes from 1,600+
239 annotated multi-agent traces; 12/14 are detectable via our span kinds. AgentDebug [Cemri et al.,
240 2025b] categorizes failures across four cognitive dimensions. Aegis [Chen et al., 2025] classifies
241 agent-environment failure modes. AgentOps [Dong et al., 2024] proposes an observability taxonomy
242 without an OTel-native implementation or empirical evaluation. These works characterize *what* goes
243 wrong but do not provide runtime telemetry to *detect* faults.

244 **Debugging tools.** AgentRx [Microsoft Research, 2026] uses LLM-based analysis of execution
245 logs for automated multi-agent diagnosis. AGDebugger [Jiang et al., 2025] provides interactive
246 visual debugging, validated through a 14-person user study. AgentDiagnose [Li et al., 2025] offers
247 systematic trajectory analysis. These tools operate on post-hoc logs; AGENTTELEMETRY provides
248 the structured telemetry layer they could consume.

249 **Safety and guardrails.** GuardAgent [Zhan et al., 2025] and NeMo Guardrails [Rebedea et al., 2023]
250 are *enforcement* mechanisms that intercept and block unsafe agent actions. Our GUARD_RAIL span
251 kind serves a complementary role: it provides *observability* for safety checks, recording whether a
252 guardrail fired, was bypassed, or failed. Enforcement without monitoring creates a blind spot.

253 **LLM observability platforms.** LangSmith [LangChain Inc., 2024] is LangChain-specific. Lang-
254 fuse [Langfuse GmbH, 2024] and Datadog LLM [Datadog Inc., 2024] capture LLM call traces but
255 do not define reusable semantic span kinds. OpenLLMetry [Traceloop, 2024] and Arize Phoenix
256 intercept API calls but cannot distinguish agent-level semantics—a planning step and a summarization
257 call are both POST /v1/chat/completions.

258 **OTel standards.** The GenAI conventions [OpenTelemetry Authors, 2024] define eight operation
259 types; AGENTTELEMETRY is a strict superset for agent orchestration, adding five novel span kinds
260 proven necessary by ablation.

261 7 Conclusion

262 We have presented AGENTTELEMETRY, the first OTel-native observability library with a grounded
263 taxonomy of agent-specific span kinds. Five of nine kinds are entirely novel, and the ablation study
264 proves all nine are necessary. The SWE-bench case study demonstrates that the novel REASONING
265 span kind characterizes the dominant failure mode (75% of instances) and enables a telemetry-guided
266 intervention improving the patch rate by +11 pp. Controlled fault injection confirms structural
267 completeness (FDR = 1.000 vs. 0.429 for vanilla OTel). Instrumentation overhead is negligible
268 ($<30\mu\text{s}$ per span).

269 **Limitations.** Core benchmarks use mock LLM clients (validated with real APIs in RQ5). Six of
270 seven framework apps use manual instrumentation stubs. The SWE-bench study uses a single model
271 and seed. The simulated user study uses LLM-generated personas, not human participants.

272 Future work includes proposing the taxonomy as an OpenTelemetry Enhancement Proposal (OTEP),
273 multi-model SWE-bench replication, and controlled human debugging studies.

274 References

- 275 M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran,
276 D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica. Why Do Multi-Agent LLM
277 Systems Fail? In *Proc. NeurIPS*, 2025.
- 278 M. Cemri, M. Y. Qiang, Y. Xiao, and L. Tan. AgentDebug: Identifying Errors in LLM-Based Agents
279 through Agent Trace Analysis. *arXiv preprint arXiv:2504.16744*, 2025.
- 280 Z. Chen, L. Xu, M. Wang, et al. Aegis: Agent-Environment Failure Mode Taxonomy and Detection.
281 *arXiv preprint arXiv:2508.19504*, 2025.

282 Datadog Inc. LLM Observability: Monitor and Improve LLM Applications. [https://www.](https://www.datadoghq.com/product/llm-observability/)
283 [datadoghq.com/product/llm-observability/](https://www.datadoghq.com/product/llm-observability/), 2024.

284 J. Dong, Y. Liu, H. Qian, L. Zheng, and L. Chen. AgentOps: An Observability Taxonomy for AI
285 Agent Operations. *arXiv preprint arXiv:2411.05285*, 2024.

286 S. Jiang, X. Wang, Y. Zhang, et al. AGDebugger: An Interactive Multi-Agent Debugging Tool. In
287 *Proc. ACM CHI*, 2025.

288 C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can
289 Language Models Resolve Real-World GitHub Issues? In *Proc. ICLR*, 2024.

290 LangChain Inc. LangSmith: Platform for Building Production-Grade LLM Applications. [https:](https://smith.langchain.com/)
291 [//smith.langchain.com/](https://smith.langchain.com/), 2024.

292 Langfuse GmbH. Langfuse: Open-Source LLM Observability Platform. <https://langfuse.com/>,
293 2024.

294 Y. Li, Z. Wang, H. Chen, et al. AgentDiagnose: A Toolkit for Diagnosing Agent Trajectories. In
295 *Proc. EMNLP (Demo Track)*, 2025.

296 X. Liu, H. Yu, H. Zhang, et al. AgentBench: Evaluating LLMs as Agents. In *Proc. ICLR*, 2024.

297 C. Ma, J. Zhang, Z. Zhu, C. Yang, Y. Yang, et al. AgentBoard: An Analytical Evaluation Board of
298 Multi-Turn LLM Agents. In *Proc. ACL*, 2024.

299 Microsoft Research. AgentRx: Automated Diagnostic Framework for Multi-Agent Systems. *arXiv*
300 *preprint arXiv:2602.02475*, 2026.

301 OpenTelemetry Authors. OpenTelemetry Specification, v1.30. [https://opentelemetry.io/](https://opentelemetry.io/docs/specs/otel/)
302 [docs/specs/otel/](https://opentelemetry.io/docs/specs/otel/), 2024.

303 A. Strauss and J. Corbin. *Basics of Qualitative Research: Techniques and Procedures for Developing*
304 *Grounded Theory*. Sage, 2nd edition, 1998.

305 Traceloop. OpenLLMetry: Open-Source Observability for LLM Applications Built on OpenTeleme-
306 try. <https://github.com/traceloop/openllmetry>, 2024.

307 X. Zhan, Y. Luo, S. Wang, et al. GuardAgent: Safeguard LLM Agents by a Guard Agent via
308 Knowledge-Enabled Reasoning. In *Proc. ICML*, 2025.

309 T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen. NeMo Guardrails: A Toolkit for
310 Controllable and Safe LLM Applications. In *Proc. EMNLP (Demo Track)*, 2023.

311 A Additional Results

312 **MAST mapping.** Table 11 maps MAST’s 14 failure modes to our fault types. 12 of 14 (85.7%) are
313 detectable; the two gaps (FM-2.2: failure to ask for clarification, FM-3.2: missing verification) are
314 omission failures requiring expected-span specifications.

Table 11: Mapping MAST failure modes to AgentTelemetry fault types.

MAST Mode	Code	Our Fault Type(s)	Cov.
Disobey Task Spec	FM-1.1	wrong_tool, hallucination	Full
Disobey Role Spec	FM-1.2	agent_misroute, GR bypass	Full
Step Repetition	FM-1.3	infinite_loop, reas. loop	Full
Loss of History	FM-1.4	ctx. overflow, mem. corr.	Full
Unaware of Term.	FM-1.5	infinite_loop, plan. fail	Full
Conversation Reset	FM-2.1	mem. corr., ctx. overflow	Partial
Fail to Clarify	FM-2.2	—	Gap
Task Derailment	FM-2.3	wrong_tool, plan. fail	Partial
Info Withholding	FM-2.4	circular_delegation	Partial
Ignored Input	FM-2.5	agent_misroute, circ. del.	Partial
Reas.-Action Mism.	FM-2.6	wrong_tool, hallucination	Full
Premature Term.	FM-3.1	timeout, plan. fail	Partial
No Verification	FM-3.2	—	Gap
Incorrect Verif.	FM-3.3	GR bypass, hallucination	Full

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and introduction state four specific contributions (taxonomy, SWE-bench case study, fault injection evaluation, open-source library), each supported by corresponding experimental sections (§3–§5).

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: Section 7 (Conclusion) includes an explicit Limitations paragraph discussing mock LLM clients, manual instrumentation stubs, single-model SWE-bench evaluation, and simulated (not human) user study.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [\[N/A\]](#)

Justification: The paper does not include theoretical results or proofs. All claims are empirically validated.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [\[Yes\]](#)

Justification: Section 5 specifies all experimental parameters (14 fault types, 5 conditions, 7 frameworks, 6 models, 2,940 configurations). The SWE-bench setup details model, iteration count, tool set, and cost. All benchmarks use deterministic mock clients for reproducibility.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [\[Yes\]](#)

Justification: The library is published on PyPI (pip install agenttelemetry) under Apache-2.0 license. All code, configurations, and mock data are open-source. An anonymized repository link will be provided in the supplementary material.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 5 provides complete experimental setup including models, frameworks, fault injection rates, privacy levels, iteration counts, and SWE-bench instance selection criteria.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: RQ1 reports FDR with standard deviations across 5 seeds. RQ6 reports 95% Wilson confidence intervals for reasoning-loop prevalence ([66%, 82%]) and closed-loop improvement across 3 seeds. The simulated user study reports Cohen’s h effect size.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Overhead benchmarks specify Apple M4 Pro hardware. SWE-bench cost is reported (\$2.53). Real LLM experiments report models and providers used.

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research involves no human subjects, no personally identifiable data, and no dual-use concerns. The library is designed to improve system reliability and transparency.

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The work improves reliability and transparency of AI agent systems (positive impact). The privacy-level controls address potential negative impacts of telemetry data collection. The observability primitives could theoretically be used for surveillance of agent behavior, but this is mitigated by the configurable privacy filtering.

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper releases an observability library, not a model or dataset with misuse risk. The library includes privacy controls to prevent unintended data exposure.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: SWE-bench is properly cited. OpenTelemetry is cited. All seven frameworks are referenced. The library uses Apache-2.0 license.

400 **13. New assets**

401 Question: Are new assets introduced in the paper well documented and is the documentation

402 provided alongside the assets?

403 Answer: [\[Yes\]](#)

404 Justification: The AgentTelemetry library is documented with API reference, installation

405 instructions, and usage examples in the repository. The span taxonomy is fully specified in

406 the paper.

407 **14. Crowdsourcing and research with human subjects**

408 Question: For crowdsourcing experiments and research with human subjects, does the paper

409 include the full text of instructions given to participants and screenshots, if applicable, as

410 well as details about compensation (if any)?

411 Answer: [\[N/A\]](#)

412 Justification: The paper does not involve crowdsourcing or human subjects. The user study

413 uses simulated (LLM-generated) personas, which is explicitly disclosed.

414 **15. Institutional review board (IRB) approvals or equivalent for research with human**

415 **subjects**

416 Question: Does the paper describe potential risks incurred by study participants, whether

417 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)

418 approvals (or an equivalent approval/review based on the requirements of your country or

419 institution) were obtained?

420 Answer: [\[N/A\]](#)

421 Justification: No human subjects were involved. The simulated user study uses LLM-

422 generated role-plays, not human participants.

423 **16. Declaration of LLM usage**

424 Question: Does the paper describe the usage of LLMs if it is an important, original, or

425 non-standard component of the core methods in this research? Note that if the LLM is used

426 only for writing, editing, or formatting purposes and does *not* impact the core methodology,

427 scientific rigor, or originality of the research, declaration is not required.

428 Answer: [\[Yes\]](#)

429 Justification: LLMs are used as experimental subjects (GPT-4o-mini for SWE-bench, 8

430 models for real API validation) and for the simulated user study (GPT-4o-mini for persona

431 role-plays). These uses are described in Sections 5.5 and 5.6.